feedforward neural network

mahmood

May 27, 2023

note
this document and others can be found on my website
`https://mahmoodsheikh36.github.io/`

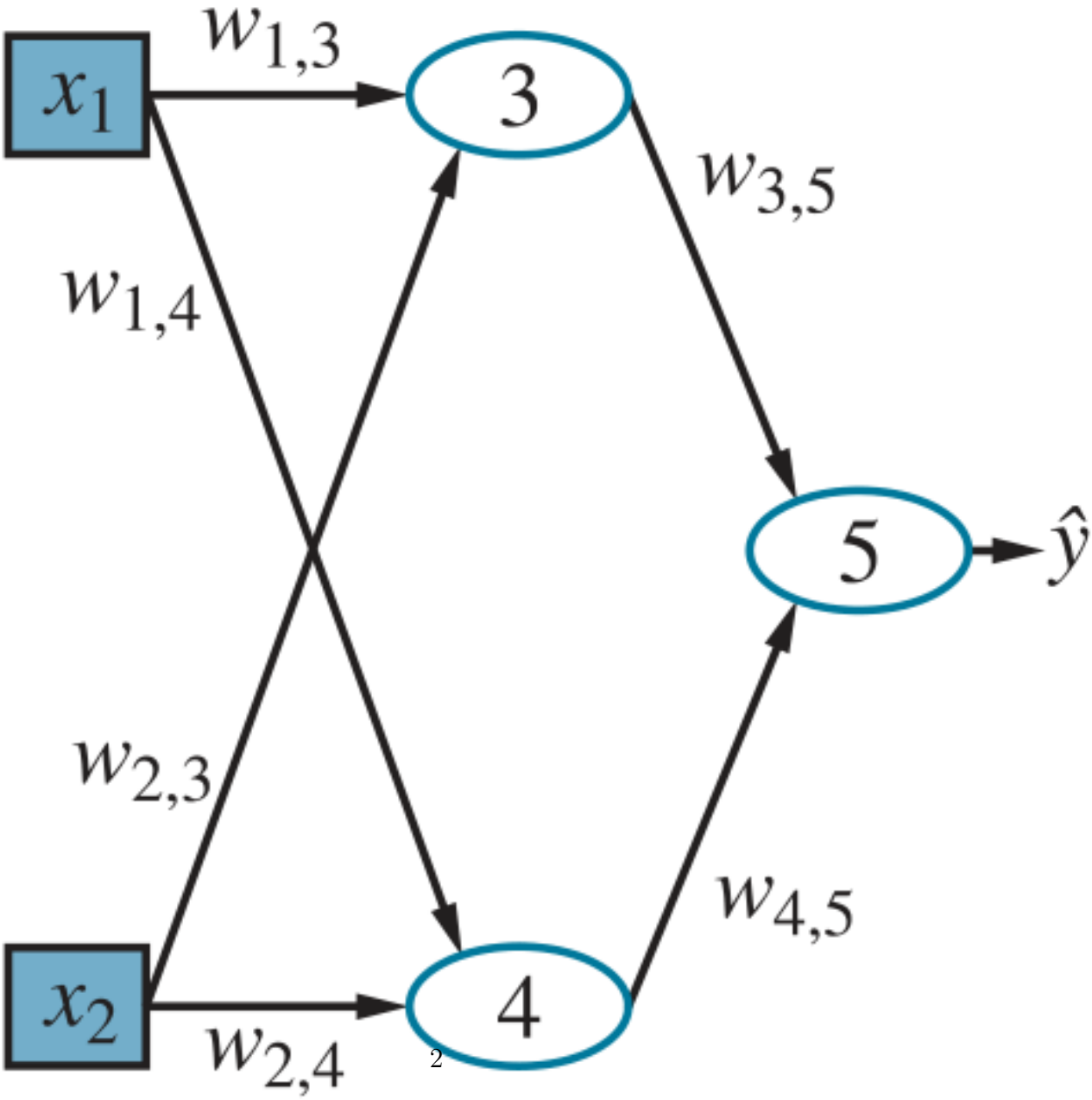
[?]
a **feedforward network** has connections only in one direction. each node computes a function of its inputs and passes the result to its successors in the network. information flows through the network from the input nodes to the output nodes, and there are no loops.
a unit calculates the weighted sum of the inputs from predecessor nodes and then applies a nonlinear function to produce its output. let a_j denote the output of unit j and let $w_{i,j}$ be the weight attached to the link from unit i to unit j; then we have

$$a_j = g_j\left(\sum_i w_{i,j} a_i\right) = g_j(in_j)$$

where g_j is a nonlinear activation function associated with unit j and in_j is the weighted sum of the inputs to unit j.
each unit has an extra input from a dummy unit 0 that is fixed to +1 and a weight $w_{0,j}$ for that input. this allows the total weighted input in_j to unit j to be nonzero even when the outputs of the preceding layer are all zero. with this convention, we can write the preceding equation in vector form:

$$a_j = g_j(\mathbf{w}^T\mathbf{x})$$

where $\mathbf{w}$ is the vector of weights leading into unit j (including $w_{0,j}$) and $\mathbf{x}$ is the vector of inputs to unit j (including the +1)


x_1
x_2
$w_{1,3}$
$w_{1,4}$
$w_{2,3}$
$w_{2,4}$
3
4
5
$w_{3,5}$
$w_{4,5}$
$\hat{y}$

consider the following network
coupling multiple units together into a network creates a complex function that is a composition of the algebraic expressions represented by the individual units. for example, the network above represents a function $h_{\mathbf{w}}(\mathbf{x})$, parameterized by weights $\mathbf{w}$, that maps a two-element input vector $\mathbf{x}$ to a scalar output value $\hat{y}$. the internal structure of the function mirrors the structure of the network. for example we can write an expression for the output $\hat{y}$ as follows:

$$\begin{aligned}\hat{y} &= g_5(in_5) = g_5(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\ &= g_5(w_{0,5} + w_{3,5}g_3(in_3) + w_{4,5}g_4(in_4)) \\ &= g_5(w_{0,5} + w_{3,5}g_3(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) + w_{4,5}g_4(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2))\end{aligned}$$

thus, we have the output $\hat{y}$ expressed as a function $h_{\mathbf{w}}(\mathbf{x})$ of the inputs and the weights.
we generally use $\mathbf{W}$ to denote a weight matrix; $\mathbf{W}^{(i)}$ denotes the weights in the i'th layer, let $\mathbf{g}^{(i)}$ denote the activation functions in the i'th layer, then, for example, an entire network of 1 input layer, 1 hidden layer, and an output node can be written as follows:

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{g}^{(2)}(\mathbf{W}^{(2)}\mathbf{g}^{(1)}(\mathbf{W}^{(1)}\mathbf{x}))$$

for the network to learn, we need to gradually adjust the weights to fit the learning data, using the gradient descent algorithm
first we apply the loss function, for simplicity the squared loss function is used here, we will calculate the gradient for the network we proposed with respect to a single training example $(\mathbf{x}, y)$. (for multiple examples, the gradient is just the sum of the gradients for the individual examples.) the network outputs a prediction $\hat{y} = h_{\mathbf{w}}(\mathbf{x})$ and the true value is y, so we have

$$Loss(h_{\mathbf{w}}) = L_2(y, h_{\mathbf{x}}(\mathbf{x})) = ||y - h_{\mathbf{w}}(\mathbf{x})||^2 = (y - \hat{y})^2$$

we compute the gradient of the loss with respect to the weights, we use the derivative chain rule, we'll start with the easy case: a weight such as $w_{3,5}$ that is connected to the output unit. we operate directly on the network-defining expressions we proposed here:

$$
\begin{aligned}
\frac{\partial}{\partial w_{3,5}} Loss(h_{\mathbf{w}}) &= \frac{\partial}{\partial w_{3,5}}(y-\hat{y})^2 = -2(y-\hat{y})\frac{\partial \hat{y}}{\partial w_{3,5}} \\
&= -2(y-\hat{y})\frac{\partial}{\partial w_{3,5}} g_5(in_5) = -2(y-\hat{y})g_5'(in_5)\frac{\partial}{\partial w_{3,5}} in_5 \\
&= -2(y-\hat{y})g_5'(in_5)\frac{\partial}{\partial w_{3,5}}(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\
&= -2(y-\hat{y})g_5'(in_5)a_3
\end{aligned}
$$

the slightly more difficult case involves a weight such as $w_{1,3}$ that is not directly connected to the output unit. here, we have to apply the chain rule one more time. the first few steps are identical, so we omit them:

$$
\begin{aligned}
\frac{\partial}{\partial w_{1,3}} Loss(h_{\mathbf{w}}) &= -2(y-\hat{y})g_5'(in_5)\frac{\partial}{\partial w_{1,3}}(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\
&= -2(y-\hat{y})g_5'(in_5)w_{3,5}\frac{\partial}{\partial w_{1,3}} a_3 \\
&= -2(y-\hat{y})g_5'(in_5)w_{3,5}\frac{\partial}{\partial w_{1,3}} g_3(in_3) \\
&= -2(y-\hat{y})g_5'(in_5)w_{3,5}g_3'(in_3)\frac{\partial}{\partial w_{1,3}} in_3 \\
&= -2(y-\hat{y})g_5'(in_5)w_{3,5}g_3'(in_3)\frac{\partial}{\partial w_{1,3}}(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) \\
&= -2(y-\hat{y})g_5'(in_5)w_{3,5}g_3'(in_3)x_1
\end{aligned}
$$

if we define $\Delta_5 = 2(\hat{y}-y)g_5'(in_5)$ as a sort of "perceived error" at the point where unit 5 receives its input, then the gradient with respect to $w_{3,5}$ is just $\Delta_5 a_3$. this makes perfect sense: if Δ_5 is positive, that means $\hat{y}$ is too big (g' is always nonnegative); if a_3 is also positive, then increasing $w_{3,5}$ will only make things worse, whereas if a_3 is negative, then increasing $w_{3,5}$ will reduce the error. the magnitude of a_3 also matters: if a_3 is small for this training example, then$w_{3,5}$ didnt play a major role in producing the error and doesnt need to be changed much.
if we also define $\Delta_3 = \Delta_5 w_{3,5} g_3'(in_3)$, then the gradient for $w_{1,3}$ becomes just $\Delta_3 x_1$. thus, the perceived error at the input to unit 3 is the perceived error at the input to unit 5, multiplied by information along the path from 5 back to 3. this phenomenon is completely general, and gives rise to the term back-propagation for the way that the error at the output is passed back through

the network.

another important characteristic of these gradient expressions is that they have as factors the local derivatives $g'_j(in_j)$. as noted earlier, these derivatives are always nonnegative, but they can be very close to zero (in the case of the sigmoid, softplus, and tanh functions) or exactly zero (in the case of ReLUs), if the inputs from the training example in question happen to put unit j in the flat operating region. if the derivative g'_j is small or zero, that means that changing the weights leading into unit j will have a negligible effect on its output. as a result, deep networks with many layers may suffer from a vanishing gradient–the error signals are extinguished altogether as they are propagated back through the network.
but how do we pass the data into the input layer? and what do the values coming out of the output layer mean? the following paragraphs explain how we **encode** data to pass it into the input layer, and how we treat the values resulting from the output layer.
the encoding of input data is usually straightforward, at least for the case of factored data where each training example contains values for n input attributes. if the attributes are boolean, we have n input nodes; usually false is mapped to an input of 0 and true is mapped to 1, although sometimes -1 and +1 are used. numeric attributes, whether integer or real-valued, are typically used as is, although they may be scaled to fit within a fixed range; if the magnitudes for different examples vary enormously, the values can be mapped onto a log scale.

images do not quite fit into the category of factored data; although an RGB image of size $X \times Y$ pixels can be thought of as $3XY$ integer-valued attributes (typically with values in the range $\{0, \ldots, 255\}$), this would ignore the fact that the RGB triplets belong to the same pixel in the image and the fact that pixel adjacency really matters. of course, we can map adjacent pixels onto adjacent input nodes in the network, but the meaning of adjacency is completely lost if the internal layers of the network are fully connected. in practice, networks used with image data have array-like internal structures that aim to reflect the semantics of adjacency.
categorical attributes with more than two values are usually encoded using the so-called **one-hot encoding**. an attribute with d possible values is represented by d separate input bits. for any given value, the corresponding input bit is set to 1 and all the others are set to 0. this generally works better than mapping the values to integers.
on the output side of the network, the problem of encoding the raw data values into actual values y for the output nodes of the graph is much the

same as the input encoding problem. for example, if the network is trying to predict a variable named weather, which has values {sun,rain,cloud,snow}, we would use a one-hot encoding with four bits.

so much for the data values $\mathbf{y}$. what about the prediction $\hat{y}$? ideally, it would exactly match the desired value $\mathbf{y}$. and the loss would be zero, and we'd be done. in practice, this seldom happens–especially before we have started the process of adjusting the weights. thus, we need to think about what an incorrect output value means, and how to measure the loss. in deriving the gradients, we began with the squared-error loss function. this keeps the algebra simple, but it is not the only possibility. in fact, for most deep learning applications, it is more common to interpret the output values $\hat{y}$ as probabilities and to use the negative log likelihood as the loss function.

maximum likelihood learning finds the value of $\mathbf{w}$ that maximizes the probability of the observed data. and because the log function is monotonic, this is equivalent to maximizing the log likelihood of the data, which is equivalent in turn to minimizing a loss function defined as the negative log likelihood. (taking logs turns products of probabilities into sums, which are more amenable for taking derivatives.) in other words, we are looking for $\mathbf{w}^*$ that minimizes the sum of negative log probabilities of the N examples:

$$\mathbf{w}^* = \arg\min_{w} - \sum_{j=1}^{N} \log P_{\mathbf{w}}(\mathbf{y}_j | \mathbf{x}_j)$$

in the deep learning literature, it is common to talk about minimizing the cross-entropy. we typically use the definition of cross-entropy with P being the true distribution over training examples, $P^*(\mathbf{x}, \mathbf{y})$, and Q being the predictive hypothesis $P_{\mathbf{w}}(\mathbf{y} \mid \mathbf{x})$. minimizing the cross-entropy $H(P^*(\mathbf{x}, \mathbf{y}), P_{\mathbf{w}}(\mathbf{y} \mid \mathbf{x}))$ by adjusting $\mathbf{w}$ makes the hypothesis agree as closely as possible with the true distribution. in reality, we cannot minimize this cross-entropy because we do not have access to the true data distribution $P^*(\mathbf{x}, \mathbf{y})$; but we do have access to samples from $P^*(\mathbf{x}, \mathbf{y})$, so the sum over the actual data in this equation approximates the expectation in the of cross-entropy.

to minimize the negative log likelihood (or the cross-entropy), we need to be able to interpret the output of the network as a probability. for example, if the network has one output unit with a sigmoid activation function and is learning a boolean classification, we can interpret the output value directly as the probability that the example belongs to the positive class. thus, for boolean classification problems, we commonly use a sigmoid output layer.

multiclass classification problems are very common in machine learning. for example, classifiers used for object recognition often need to recognize thousands of distinct categories of objects. natural language models that try to

predict the next word in a sentence may have to choose among tens of thousands of possible words. for this kind of prediction, we need the network to output a categorical distribution–that is, if there are d possible answers, we need d output nodes that represent probabilities summing to 1.
To achieve this, we use a softmax layer, which outputs a vector of d values given a vector of input values $\mathbf{in} = (in_1, \ldots, in_d)$. The kth element of that output vector is given by

$$\text{softmax}(\mathbf{in})_k = \frac{e^{in_k}}{\sum_{k'=1}^{d} e^{in_{k'}}}$$

for a regression problem, where the target value y is continuous, it is common to use a linear output layer–in other words, $\hat{y}_j = in_j$, without any activation function g.
many other output layers are possible.
training a neural network consists of modifying the networks parameters so as to minimize the loss function on the training set. in principle, any kind of optimization algorithm could be used. in practice, modern neural networks are almost always trained with some variant of stochastic gradient descent.
here, the goal is to minimize the loss $L(\mathbf{w})$, where $\mathbf{w}$ represents all of the parameters of the network (all the weights). each update step in the gradient descent process looks like this:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w})$$

note

the upside down delta is called **nabla** actually, and it refers to computing the descent of the function $L(\mathbf{w})$ with respect to $\mathbf{w}$
this notation sucks butt, i've been told its mostly like that in CS books, the loss function shouldnt be taking weights anyway, it should probably be $L(h(\mathbf{w}))$ where h is the network itself
because $f(g(x))$ isnt the same as $f(x)$ even though both functions depend on x

where α is the learning rate. for standard gradient descent, the loss L is defined with respect to the entire training set. for stochastic gradient descent, it is defined with respect to a minibatch of m examples chosen randomly at each step.

overall, the process of learning the weights of the network is usually one that exhibits diminishing returns. we run until it is no longer practical to decrease the test error by running longer. usually this does not mean we have reached a global or even a local minimum of the loss function. instead, it means we would have to make an impractically large number of very small steps to continue reducing the cost, or that additional steps would only cause overfitting, or that estimates of the gradient are too inaccurate to make further progress.

1 common lisp implementation

this code uses common lisp (aswell as code/functions from the actual link itself, click it for those)

```
;; neural network class
(defclass network ()
  ((weights :initarg :weights :initform nil :accessor
     network-weights)
   (input-layer-size :initarg :input-layer-size :accessor
      network-input-layer-size)
   (output-layer-size :initarg :output-layer-size :accessor
      network-output-layer-size)
   (hidden-layer-sizes :initarg :hidden-layer-sizes :accessor
      network-hidden-layer-sizes)
   (learning-rate :initform 0.0005 :initarg :learning-rate :
      accessor network-learning-rate)))

(defmethod network-feedforward ((nw network) x)
  "pass the vector x into the network, x is treated as the
     input layer, the activations of the entire network are
     returned"
  (let ((previous-layer-activations x)
        (network-activations (list x))
        (network-unsquashed-activations (list x)))
    (loop for weights-index from 0 below (length (
       network-weights nw))
         do (let* ((weights (elt (network-weights nw)
            weights-index))
                  (layer-activations (list->vector (make-list (
```

```
                      length (elt weights 0)) :initial-element
                      0))))
              (loop for i from 0 below (length
                 previous-layer-activations)
                   do (let ((previous-unit-activation (elt
                       previous-layer-activations i))
                            (previous-unit-weights (elt weights
                               i)))
                        (setf
                         layer-activations
                         (map
                          'list
                          (lambda (weight activation)
                            (+ activation (*
                               previous-unit-activation weight)
                               ))
                          previous-unit-weights
                             layer-activations))))
              (setf previous-layer-activations (map 'list #'
                 sigmoid layer-activations))
              (setf network-activations
                   (append network-activations
                          (list previous-layer-activations)))
              (setf network-unsquashed-activations
                   (append network-unsquashed-activations
                          (list layer-activations)))))
    (values (list->vector network-activations) (list->vector
       network-unsquashed-activations))))

(defun sigmoid (x)
  (cond ((< x -70) 0) ;; to avoid float overflow
        (t (/ 1 (1+ (exp (- x)))))))

(defun sigmoid-derivative (x)
  (* (sigmoid x) (- 1 (sigmoid x))))

(defmethod network-generate-weights ((nw network))
  "generate random weights based on the sizes of the layers"
  ;; weights from input layer to first hidden layer
  (setf
   (network-weights nw)
```

```
  (append
   (network-weights nw)
   (list (loop for i from 0 below (network-input-layer-size nw
      )
               collect (loop for j from 0 below (car (
                  network-hidden-layer-sizes nw))
                           collect (generate-random-weight)))))
                              )
 ;; weights between hidden layers
 (setf
  (network-weights nw)
  (append
   (network-weights nw)
   (loop for i from 0 below (1- (length (
      network-hidden-layer-sizes nw)))
        collect (let ((hidden-layer-size
                         (elt (network-hidden-layer-sizes nw) i)
                            )
                       (next-hidden-layer-size
                         (elt (network-hidden-layer-sizes nw)
                            (1+ i))))
                  (loop for j from 0 below hidden-layer-size
                      collect (loop for k from 0 below
                         next-hidden-layer-size
                                  collect (
                                     generate-random-weight)))
                                     ))))
 ;; weights from last hidden layer to output layer
 (setf
  (network-weights nw)
  (append
   (network-weights nw)
   (list (loop for i from 0 below (elt
                                 (network-hidden-layer-sizes nw
                                    )
                                 (1- (length (
                                    network-hidden-layer-sizes
                                    nw))))
               collect (loop for j from 0 below (
                  network-output-layer-size nw)
                           collect (generate-random-weight)))))
```

```
                                )
  ;; convert weights from nested lists to nested vector for
     fast access
  (setf (network-weights nw) (list->vector (network-weights nw)
     )))

(defmethod print-object ((nw network) stream)
  (print-unreadable-object (nw stream :type t)
    (format stream "total weights: ~a"
            (reduce
             #'*
             (append (list (network-input-layer-size nw) (
                network-output-layer-size nw))
                    (network-hidden-layer-sizes nw))))))

(defun generate-random-weight ()
  (/ (- (random 2000) 1000) 1000))

(defun make-network (&key input-layer-size hidden-layer-sizes
   output-layer-size
                     learning-rate)
  (let ((nw (make-instance 'network
                        :input-layer-size input-layer-size
                        :hidden-layer-sizes hidden-layer-sizes
                        :output-layer-size output-layer-size
                        :learning-rate learning-rate)))
    (network-generate-weights nw)
    nw))
```

while implementing backpropagation i faced a challenge(s), can i simply iterate through the layers one by one like i did in `network-feedforward` and just handle them each at a time? at first i thought i couldnt, because i need to go through each possible path from the output layer to the input layer and update each weight, each weight's gradient depends on the gradient for the weight preceding it in the path, but after alot of thinking i realized that i could just store the gradients of each matrix of weights between 2 layers and update the gradient matrix as we go, this would probably work i think, but im more curious about how i would go about finding all paths and operating on them in the way i described.
we have a variable number of layers n, we definitely could iterate through these layers but on each iteration we only have access to the current layers

units, and to construct a path from the input layer to the output layer we need to find n units, one from each layer, so this cannot be done by a simple for loop that iterates through the layers one at a time, what about nested loops (one for each layer)? but how are we supposed to nest n loops? this isnt possible, so maybe recursion could help?
say we did use recursion, and on each recurrence we hopped backwards into the previous layer in the network, how are we supposed to return the different units from a layer to a function call? well we dont need to, we could just call the function on the many units we have in that previous layer and pass it an index to say which unit we're referring to.

```
(defmethod network-train ((nw network) xs ys)
  "train on the given data, xs is a list of vectors, each
     vector is treated as an input layer to the network, and ys
      is a list of vectors, each vector representing the output
      layer corresponding to the vector in xs thats at the same
      index"
  (loop for i from 0 below (length xs)
        do (let*
               ((x (elt xs i))
                (y (elt ys i))
                (layer-index (1+ (length (
                   network-hidden-layer-sizes nw)))))
            (multiple-value-bind (activations
               unsquashed-activations)
                (network-feedforward nw x)
              (loop
                for unit-index
                from 0
                  below (network-output-layer-size nw)
                do (network-back-propagate
                    nw
                    layer-index
                    unit-index
                    (* 2 (- (elt (elt activations layer-index)
                       unit-index)
                           (elt y unit-index)))
                    activations
                    unsquashed-activations))))))
```

```
(defmethod network-back-propagate ((nw network) layer-index
   unit-index gradient activations unsquashed-activations)
  "backpropagate the error through the network, each layers
     gradients depend on those of the layer succeeding it in
     the network"
  (let*
      ((predecessor-layer-weights (elt (network-weights nw) (1-
          layer-index)))
       (weights-into-unit
         (map
          'vector
          (lambda (predecessor-unit-weights)
            (elt predecessor-unit-weights unit-index))
          predecessor-layer-weights))
       (unit-activation (elt (elt activations layer-index)
          unit-index))
       (unit-input (elt (elt unsquashed-activations layer-index
          ) unit-index)))
    (loop for predecessor-unit-index from 0 below (length
       predecessor-layer-weights)
          do (let*
                 ((weight (elt weights-into-unit
                    predecessor-unit-index))
                  (predecessor-unit-activation
                    (elt (elt activations (1- layer-index))
                         predecessor-unit-index))
                  (gradient-to-back-propagate
                    (* gradient
                       (sigmoid-derivative unit-input)
                       weight))
                  (actual-gradient (* gradient
                                      (sigmoid-derivative
                                         unit-input)
                                      predecessor-unit-activation)
                                         )
                  (weight-change (* (network-learning-rate nw)
                                    actual-gradient)))
                (if (> layer-index 1)
                    (network-back-propagate
                     nw
                     (1- layer-index)
```

```
                    predecessor-unit-index
                    gradient-to-back-propagate
                    activations
                    unsquashed-activations))
               (setf
                (elt (elt (elt (network-weights nw) (1-
                   layer-index))
                         predecessor-unit-index)
                 unit-index)
                (- weight weight-change))))))
```

lets try training on some simple data

```
(defparameter nw (make-network
                 :input-layer-size 3
                 :hidden-layer-sizes '(2)
                 :output-layer-size 1
                 :learning-rate 0.01))
(loop for i from 0 below 10000
     do (network-train nw '((0 0 1) (1 1 1) (1 0 1) (0 1 1))
        '((0) (1) (1) (0))))
(multiple-value-bind (activations unsquashed-activations)
   (network-feedforward nw '(1 1 0))
  (print activations))
(multiple-value-bind (activations unsquashed-activations)
   (network-feedforward nw '(0 1 0))
  (print activations))
```

```
#(#(1 1 0) #(0.054582246 0.9765959) #(0.97331196))
#(#(0 1 0) #(0.5611936 0.525953) #(0.43829834))
```

it predicts correctly that the first number is the output number
using this code, i was getting 125k weight updates in 0.01 seconds (only 1 thread, no gpu), which i think is as far as i can get in terms of efficiency without some more advanced matrix algorithms, i tried python and got 1m (simple addition) operations in 0.04 seconds:

```
~  time python -c '
i=1
for i in range(999999):
```

```
  i = i + 1
print(i)'
999999
python -c ' i=1; for i in range(999999): i = i + 1; print(i)'
   0.04s user 0.00s system 98% cpu 0.045 total
```

we need a method to keep to keep track of loss, aswell as visual feedback would be nice, see common lisp graphics which ill be also using here

```
(defmethod network-test ((nw network) xs ys)
  "test the given data (collection of vectors for input/output
     layers), return the total loss"
  (let ((loss 0))
    (loop for i from 0 below (length xs)
          do (let*
                 ((x (elt xs i))
                  (y (elt ys i)))
               (multiple-value-bind (activations
                  unsquashed-activations)
                  (network-feedforward nw x)
                 (let* ((output-layer (elt activations (1- (
                    length activations))))
                       (example-loss (expt (vector-sum (
                          vector-sub y output-layer)) 2)))
                  (incf loss example-loss)))))
    loss))
```

we can use this function to plot the loss function after each epoch:

```
(defparameter *train-in* '((0 0 1) (1 1 1) (1 0 1) (0 1 1)))
(defparameter *train-out* '((0) (1) (1) (0)))
(defparameter *win* (make-instance 'window :draw-axis t :width
   800 :height 800))
(defparameter *nw* (make-network
                    :input-layer-size 3
                    :hidden-layer-sizes '(2)
                    :output-layer-size 1
                    :learning-rate 0.01))
(let* ((loss-history '())
       (epochs 10000)
       (axis (make-axis
```

```
            :min-x (/ (- epochs) 10)
            :max-x epochs
            :pos (make-point-2d :x 0 :y 0)
            :width (window-width *win*)
            :height (window-height *win*))))
  (loop for i from 0 below epochs
        do (network-train *nw* *train-in* *train-out*)
           (let ((loss (network-test *nw* *train-in* *train-out
              *)))
             (setf loss-history (append loss-history (list loss)
                ))
             (if (> loss (axis-max-y axis))
                 (setf (axis-max-y axis) (* loss 1.1)))))
                 ;; (setf (axis-min-y axis) (* loss 0.1)))))
  (let* ((loss-plot (make-discrete-plot
                  :points (map
                          'vector
                          (lambda (loss epoch) (make-point-2d
                             :x epoch :y loss))
                          loss-history
                          (loop for i from 0 below epochs
                             collect i)))))
    (axis-add-renderable axis loss-plot)
    (window-add-renderable *win* axis)
    (window-run *win*)))
```

although notice that here we're testing the network on the same data we trained it with, which generally isnt a good measure of the performance of the network, but this is just an example of how we can keep track of a property